

CH08 前端模块化—复习要点

1 前端独立开发与模块化的由来

1.1 前后端分离后的前端挑战

[p.7–11]

- > 前后端分离后，前端独立开发暴露的核心问题 [p.7–9]:
 - 必须手动引入所有 JS 文件，且引入顺序很重要 [p.9]
 - 全局变量污染、命名冲突风险、函数也是全局的 [p.10]
 - JS 文件之间的依赖关系难以管理 [p.10]
- > 从三个维度分析问题 [p.11]:
 - 代码质量：繁琐、可读性差、代码质量低、不便于复杂业务逻辑
 - 效率：一个页面中可能多次异步请求服务器，服务器压力大
 - 架构：无开发流程、构建规范，难以重用
- > 对比 Java 后端开发（有 Maven、分包、接口等成熟工程化体系），前端亟需工程化 [p.11]

1.2 前端工程化与模块化概念

[p.12–13]

- > 工程化 (Engineering) [p.12]: 广义上属于软件工程，包含四个方面：
 - 模块化 (Modularization): 代码拆分与组装
 - 组件化 (Componentization): UI 单元封装复用
 - 规范化 (Standardization): 代码规范、目录结构
 - 自动化 (Automation): 构建、测试、部署自动化
- > 解决的问题：可维护性、可复用性、可扩展性 [p.12]
- > 模块化定义 [p.12]: 将大工程拆分成相互依赖的小工程，编译或运行时再统一拼装和加载。狭义前端工程化有时特指模块化
- > 模块化分类 [p.13]:
 - 外部模块化：引入第三方包/插件，由 Node.js 和 NPM 管理
 - 内部模块化：工程内部的分层分类，由 CommonJS、ES6 Module 和构建工具组织
- > 模块化主要内容 [p.13]:
 - 外部模块管理：Node.js + NPM
 - 内部模块组织：CommonJS、ES6 Module + 构建工具
 - 编译转换：Babel 等转译工具
- > 诞生的大量技术：ES6、Node.js、NPM、CommonJS、Webpack、Vite、Angular、React、Vue 等 [p.12]

2 Node.js

2.1 Node.js 简介

[p.15–19]

- > Node.js 是跨平台、开源的**后端 JavaScript 运行环境**，运行在 Chrome V8 引擎上 [p.15]
- > **Node.js = JavaScript 运行时 + 生态系统** [p.16]:
 - **V8 引擎**：来自 Chrome 浏览器，让 JavaScript 脱离浏览器运行
 - **事件驱动、非阻塞 I/O**：高性能，适合处理大量并发请求
 - **NPM (Node Package Manager)**：包管理器，管理第三方依赖
 - **内置模块**：`fs`（文件系统）、`http`（HTTP 服务器）、`path`（路径处理）等
- > **Node.js 对前端的意义** [p.17]:
 1. **提供工程化基础**：本地运行环境、包管理器 (NPM)、脚本工具 (`package.json`)、模块系统 (CommonJS)
 2. **催生现代工具链**：构建工具 (Webpack、Vite)、框架 (Vue、React、Angular)
 3. **统一语言栈**：前后端都可以用 JavaScript 开发
- > 官网：<https://nodejs.org> [p.18]
- > 建议使用版本管理工具管理 Node.js 版本 [p.19]

3 NPM 包管理器

3.1 NPM 简介

[p.20–23]

- > **NPM (Node Package Manager)** [p.20]：Node.js 包管理工具，内置于 Node.js 环境
- > 两大功能 [p.20]:
 - **包仓库 (Registry)**：云端包存储仓库
 - **CLI 命令行工具**：安装、卸载、管理包
- > 初始化：`npm init` → 生成 `package.json` 文件 [p.20]
- > 语义化版本 (Semantic Versioning) [p.21]：`major.minor.patch`
- > 常用命令 [p.22]:
 - `npm install jquery` — 安装最新版，添加到 dependencies
 - `npm install jquery@2.1.x` — 安装指定版本
 - `npm install --save-dev eslint` 或 `-D` — 开发依赖 (devDependencies)，生产环境不包含
 - `npm install -g webpack` — 全局安装（命令行工具常用）
 - 安装后自动生成 `package-lock.json` 锁定版本
- > 其他包管理工具 [p.23]：Yarn、pnpm 等

3.2 Yarn

[p.24–25]

- > Facebook、Google 等联合推出的 JavaScript 包管理工具 [p.24]
- > 主要弥补 npm 的两个缺陷 [p.24]:
 - 安装速度慢（特别是大项目）
 - 版本一致性问题
- > Yarn 的改进 [p.24]:
 - **并行安装 + 本地缓存**提高安装速度
 - `yarn.lock` 文件锁定包版本
 - 更简洁的输出信息和语义化的命令

4 内部模块化的演进

4.1 原始阶段—函数作用域封装

[p.27–29]

- > 最初方式 [p.27]: 在 HTML 中一一引入不同 JS 文件, 每个文件代表一个模块
- > 改进 [p.28]: 将每个模块包裹在函数作用域内执行, 避免污染全局环境。通过匿名函数返回模块输出供其他模块使用
- > 遗留问题 [p.29]:
 - 项目扩大后 HTML 中大量 `<script>` 标签
 - Script 顺序不能契合模块间的树状/网状依赖关系
 - 代码逻辑关系难以理解, 不便于维护
 - 同步加载容易导致页面卡死
 - 仍可能因全局变量污染导致命名冲突

4.2 在线处理阶段—AMD / CMD

[p.30–32]

- > **AMD** (Asynchronous Module Definition) [p.30]: 代表 `require.js`
- > **CMD** (Common Module Definition) [p.30]: 代表 `sea.js`
- > 主要思想 [p.30]: 提供 API 和语法声明模块及依赖关系, 浏览器下载 JS 后根据声明分析依赖逐步加载 (在线编译)
- > 问题 [p.30]:
 - 在线组织模块延长前端页面加载时间, 影响用户体验
 - 加载过程中发出大量 HTTP 请求, 降低页面性能

4.3 预处理阶段—CommonJS / ES Module

[p.33–38]

- > 当前主流方案, 在代码预处理阶段完成模块化 [p.33]
- > 思想 [p.33]: 提供特殊语法, 用工具 (Webpack) 提前合并代码, 部署前完成处理
 - 节约页面加载时间 (不再在线编译)
 - 多个 script 合并为少数几个, 减少 HTTP 请求
- > **CommonJS** [p.34–37]:
 - 每个文件是一个模块, 有自己的作用域, 变量/函数/类私有
 - `module.exports` 是对外接口, `require()` 加载模块
 - `module` 是当前模块对象, `exports` 属性暴露接口
 - 模块首次加载后缓存运行结果, 后续加载直接读缓存 [p.37]
 - 加载顺序按代码中出现顺序 [p.37]
- > **ES Module** [p.38]:
 - ES6 之前 JavaScript 没有自己的模块规范, 社区制定 CommonJS
 - ECMAScript 标准委员会借鉴 CommonJS 思想写入 ES6 语法 (官方标准)
 - 语法简洁优雅: `export / import`
 - 所有现代浏览器默认支持

4.4 模块化演进对比

阶段	方式	代表	核心问题
原始	函数包裹	script 标签引入	依赖顺序、全局污染 [p.27-29]
在线处理	AMD/CMD	require.js/sea.js	大量 HTTP 请求、性能差 [p.30]
预处理	CommonJS/ESM	Node/ES6	需要构建工具支持 [p.33-38]

5 构建打包工具

5.1 为什么需要构建工具？[p.40–41]

- > 前端模块化目标：开发模块化而非运行模块化 [p.40]
- > 两步流程 [p.40]:
 1. 通过 CommonJS 或 ES6 实现 JavaScript 模块化
 2. 通过 Webpack/Vite 等工具编译处理：合并请求、去重依赖、体积优化、兼容性处理、导入 npm 包
- > 同时处理其他资源（HTML、CSS、图片等）的模块化 [p.40]
- > 构建工具的三大角色 [p.41]:
 - 翻译官：语法转换（如 ES6→ES5、TypeScript→JS）
 - 优化师：压缩、合并、Tree Shaking
 - 搬运工：资源处理、文件复制

5.2 Webpack[p.42–53]

- > 前端资源加载和打包工具，根据模块依赖关系静态分析，按规则生成静态资源 [p.42]
- > 可将 js、css、html 等转换成一个静态文件，减少页面请求 [p.42]
- > 五大核心概念：
 - 入口 (Entry) [p.44]：指示构建依赖图的起始模块，在 `webpack.config.js` 的 `entry` 属性配置
 - 输出 (Output) [p.45]：指定 bundles 的输出目录和命名规则，`output` 属性配置
 - 加载器 (Loader) [p.46]：webpack 自身只理解 JavaScript，非 JS 资源需 loader 转换。如 `css-loader + style-loader` 处理 CSS
 - 插件 (Plugin) [p.47]：执行比 loader 范围更广的任务（打包优化、压缩、环境变量）。需 `require` 后添加到 `plugins` 数组。如 `HtmlWebpackPlugin`
 - 模式 (Mode) [p.48]：`development`（开发）vs `production`（生产，会加密压缩）
- > 完整构建项目范例 [p.49–53]：从配置到输出的完整流程

5.3 Vite[p.54]

- > 新一代前端构建工具，利用浏览器原生 ES Module 支持
- > 相比 Webpack：开发服务器启动更快、热更新更迅速
- > 开发环境无需打包，生产环境使用 Rollup 打包

6 本章小结[p.55]

- > 模块化由来 [p.7–13]：前后端分离的挑战 → 工程化四方面 → 外部/内部模块化
- > Node.js [p.15–19]：V8 引擎、事件驱动、非阻塞 I/O、对前端三大意义
- > NPM [p.20–25]：包仓库 +CLI、语义化版本、常用命令、Yarn 改进

- > 模块化演进三阶段 [p.27-38]: 原始 (函数包裹)→ 在线 (AMD/CMD)→ 预处理 (CommonJS/ESM)
- > **Webpack** [p.42-53]: 五大核心 (Entry/Output/Loader/Plugin/Mode)、Vite 新一代工具 [p.54]

高频考点: (1) 前端工程化四方面 [p.12]; (2) Node.js 对前端三大意义 [p.17]; (3) 模块化三阶段演进及各阶段代表技术 [p.27-38]; (4) CommonJS (`module.exports/require`) vs ES Module (`export/import`) [p.34, p.38]; (5) Webpack 五大核心概念 [p.44-48]; (6) NPM 常用命令及 `dependencies` vs `devDependencies` [p.22]; (7) Yarn vs npm 的三大改进点 [p.24]